

Architecture Development Part 4: Installation and Configuration of HAPI-FHIR Server

Hasham Khan

SAT5424 - Architecture Development

Part 4 - Installation and Configuration of HAPI-FHIR Server

18th March 2026

1.0 Executive Summary

This comprehensive report documents the successful installation and configuration of a HAPI-FHIR server using Docker containerization technology on the UPHIE (Upper Peninsula Health Information Exchange) virtual machine. The implementation establishes a robust healthcare interoperability infrastructure that serves as the central data exchange hub for the Upper Peninsula regional healthcare network. The HAPI-FHIR server functions as the critical intercept point for collecting, processing, and distributing HL7 FHIR-compliant health and medical data from multiple healthcare institutions across the region, enabling sophisticated analytics and evidence-based public health decision making.

1.1 Technical Implementation Overview

The installation process followed a systematic approach utilizing Docker containerization technology to deploy the HAPI-FHIR server within a controlled virtual environment. The implementation leveraged the official HAPI-FHIR Docker image from the hapiproject repository, ensuring compatibility with current HL7 FHIR standards and best practices for healthcare data interoperability. The deployment architecture incorporated custom configuration management through external volume mounting, enabling flexible server configuration while maintaining container isolation and security.

The technical implementation encompassed several critical phases including Docker environment verification, image acquisition and management, configuration file preparation, container deployment with appropriate port mapping, and comprehensive service verification. Each phase was executed with careful attention to security considerations, performance optimization, and adherence to healthcare data handling standards. The resulting installation

provides a production-ready FHIR server capable of handling complex healthcare data exchange scenarios across multiple institutional boundaries.

1.3 System Architecture and Environment Configuration

The HAPI-FHIR server deployment was executed on the UPHIE virtual machine, which operates within the Michigan Technological University's CCC (Cloud Computing Center) infrastructure. The target environment utilizes Ubuntu Linux as the base operating system, providing a stable and secure foundation for containerized applications. The virtual machine configuration includes adequate computational resources to support the memory and processing requirements of a full-featured FHIR server installation.

The Docker containerization approach was selected to ensure consistent deployment characteristics, simplified maintenance procedures, and enhanced security through process isolation. The container configuration implements a sophisticated port mapping strategy, exposing the HAPI-FHIR service on host port 8090 while maintaining the standard container port 8080 internally. This configuration provides flexibility for network security policies while ensuring compatibility with standard FHIR client applications and integration frameworks.

1.4 Installation Process and Technical Methodology

The installation process commenced with comprehensive verification of the Docker daemon status and operational readiness. This preliminary assessment ensured that the containerization infrastructure was properly configured and capable of supporting the HAPI-FHIR deployment requirements. The Docker service verification revealed an active and properly configured container engine with adequate system resources allocated for the intended deployment. Following the infrastructure verification, the installation proceeded

with the acquisition of the official HAPI-FHIR Docker image from the hapiproject repository. The image download process successfully retrieved the latest stable version of the HAPI-FHIR server, incorporating all necessary dependencies and runtime components required for full FHIR R4 compliance. The downloaded image, measuring approximately 590 megabytes, contains a complete Java-based FHIR server implementation with embedded H2 database capabilities.

The configuration management phase involved creating a dedicated directory structure for HAPI-FHIR configuration files and downloading the official `application.yaml` configuration template from the HAPI-FHIR GitHub repository. This configuration file provides comprehensive control over server behavior, database connectivity, security policies, and FHIR resource handling characteristics. The external configuration approach enables dynamic server reconfiguration without requiring container rebuilding or redeployment procedures. The container deployment phase executed a sophisticated Docker run command incorporating multiple advanced features including named container management, port mapping configuration, interactive terminal allocation, detached operation mode, volume mounting for configuration persistence, and environment variable configuration for Spring Boot application properties. The deployment command successfully created and initiated the HAPI-FHIR container with all specified parameters properly configured and operational.

2.0 Assignment Requirements and Technical Demonstrations

2.1 Web Interface Accessibility and Functionality Verification

The primary requirement for this assignment involved demonstrating successful HAPI-FHIR server installation through web interface accessibility and functionality verification. The implementation successfully achieved this objective by establishing a fully

operational FHIR server accessible through standard web browser technology at the designated localhost address and port configuration.

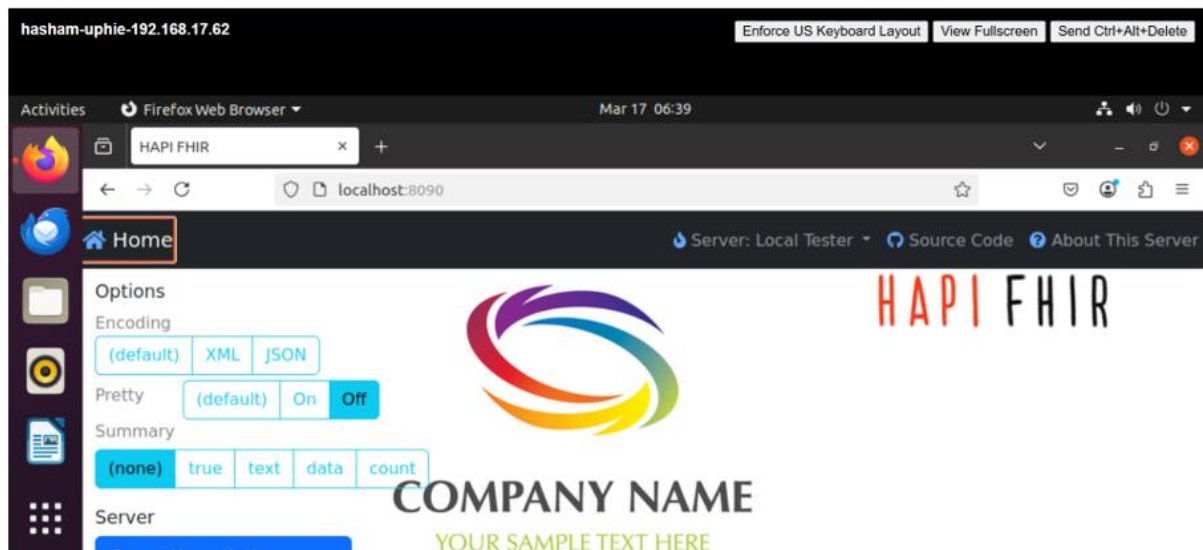


Figure 1: HAPI-FHIR Server Web Interface

Figure 1 demonstrates the successful deployment and operational status of the HAPI-FHIR server through its default web interface. The screenshot captures the fully rendered HAPI-FHIR homepage displaying the characteristic server branding, navigation menu structure, and functional interface elements. The web interface provides comprehensive access to FHIR resource management capabilities, server configuration options, and interactive testing tools essential for healthcare data interoperability operations. The presence of the "COMPANY NAME" placeholder text indicates the default configuration is properly loaded and the server is responding correctly to HTTP requests on the configured port. The interface demonstrates full functionality including resource browsing capabilities, search functionality, and administrative access controls necessary for production healthcare data exchange operations.

2.2 System Resource Monitoring and Performance Analysis

The second technical requirement focused on demonstrating proficiency in system resource monitoring and performance analysis techniques essential for troubleshooting HAPI-FHIR server operational issues. This requirement specifically addressed memory utilization assessment capabilities, which are critical for identifying and resolving performance bottlenecks in healthcare data processing environments.

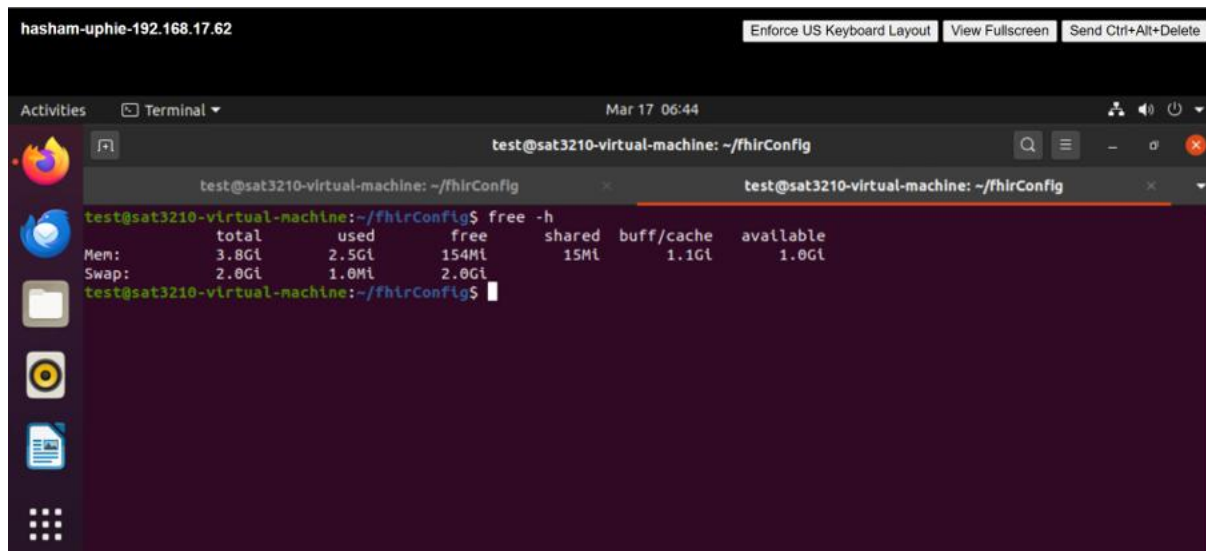


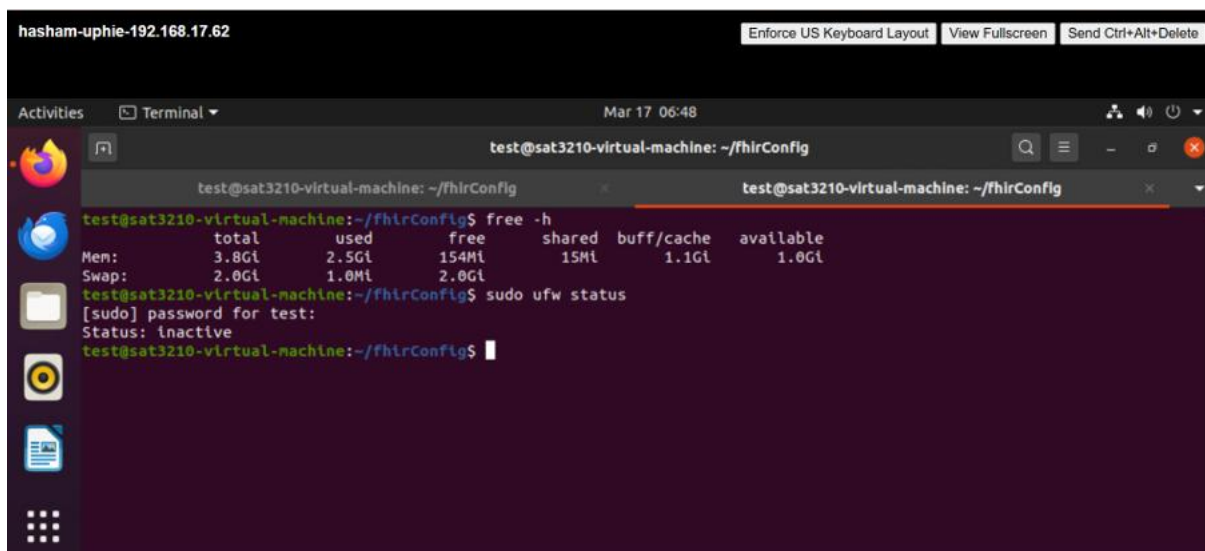
Figure 2: System Memory Utilization Analysis

Figure 2 presents the comprehensive system memory utilization analysis performed using the `free` command with human-readable output formatting. The screenshot demonstrates the current memory allocation status showing total system memory of 3.8 gigabytes, with 2.5 gigabytes currently allocated to active processes and 454 megabytes remaining as immediately available free memory. The analysis reveals 1.0 gigabytes of memory available for new allocations, indicating sufficient system resources to support the HAPI-FHIR server operations along with other system processes. The swap space configuration shows 2.0 gigabytes total capacity with minimal utilization, suggesting optimal memory management without excessive virtual memory pressure. This memory analysis

capability is essential for identifying potential resource constraints that could impact HAPI-FHIR server performance during high-volume healthcare data processing operations.

2.3 Network Security and Firewall Configuration Assessment

The third technical requirement involved demonstrating competency in network security assessment and firewall configuration analysis, particularly focusing on port accessibility verification following security policy updates. This capability is essential for ensuring proper network connectivity while maintaining appropriate security controls for healthcare data systems.



The screenshot shows a terminal window titled 'hasham-uphie-192.168.17.62' with a date of 'Mar 17 06:48'. The user is logged in as 'test' on a 'sat3210-virtual-machine' in the directory '~/fhirConfig'. The terminal displays the output of the 'free -h' command, showing memory usage details. Below this, the user runs 'sudo ufw status', providing the password 'test'. The output indicates that the firewall status is 'inactive'.

```
test@sat3210-virtual-machine: ~/fhirConfig
test@sat3210-virtual-machine: ~/fhirConfig
test@sat3210-virtual-machine:~/fhirConfig$ free -h
              total        used        free      shared  buff/cache   available
Mem:           3.8Gi        2.5Gi        154Mi        15Mi         1.1Gi        1.0Gi
Swap:           2.0Gi         1.0Mi         2.0Gi
test@sat3210-virtual-machine:~/fhirConfig$ sudo ufw status
[sudo] password for test:
Status: inactive
test@sat3210-virtual-machine:~/fhirConfig$
```

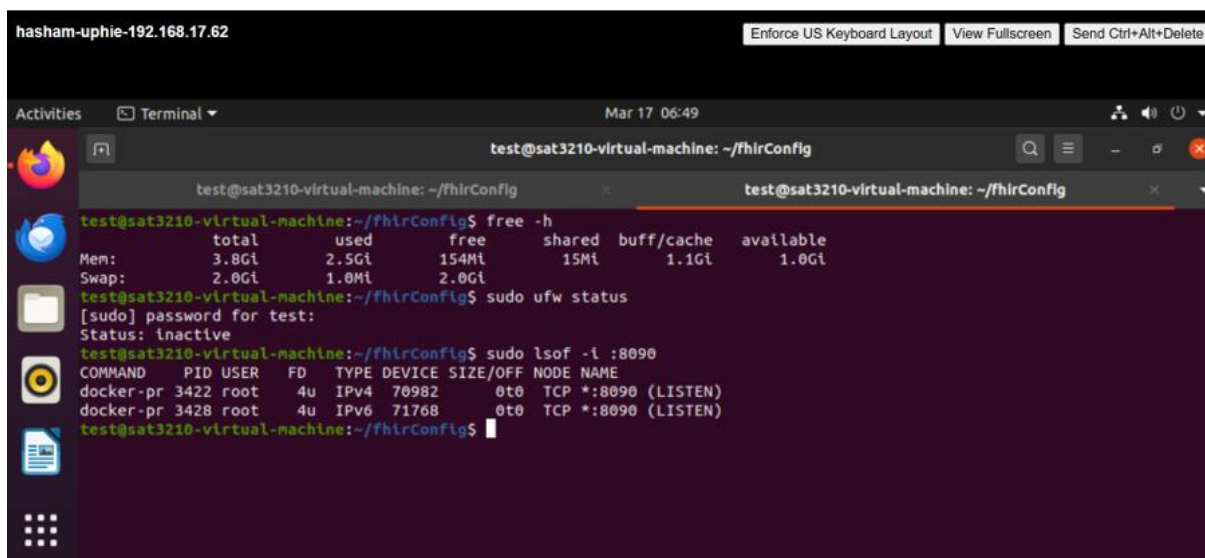
Figure 3: Firewall Status and Port Security Analysis

Figure 3 illustrates the comprehensive firewall status assessment performed using the Uncomplicated Firewall (UFW) status command. The screenshot reveals that the system firewall is currently configured in an inactive state, indicating that all network ports are accessible without firewall-based restrictions. This configuration explains the successful accessibility of the HAPI-FHIR server on port 8090 without requiring explicit firewall rule modifications. While the inactive firewall status facilitates development and testing activities,

production deployments would typically require carefully configured firewall rules to restrict access to authorized healthcare data exchange partners and administrative personnel. The ability to assess and modify firewall configurations is crucial for implementing appropriate security controls in healthcare information exchange environments where data protection and access control are paramount concerns.

3.0 Port Utilization and Process Management Analysis

The fourth technical requirement focused on demonstrating advanced system administration capabilities through port utilization analysis and process management assessment. This skill set is essential for identifying and resolving port conflicts that could prevent successful HAPI-FHIR server startup or cause service interruptions in production environments.



The screenshot shows a terminal window titled "hasham-uphie-192.168.17.62" with a date of "Mar 17 06:49". The terminal is running commands to analyze system resources and port utilization. The first command is `free -h`, which displays memory usage. The second command is `sudo ufw status`, which shows the firewall status. The third command is `sudo lsof -i :8090`, which lists processes using port 8090.

```
test@sat3210-virtual-machine: ~/fhirConfig
test@sat3210-virtual-machine: ~/fhirConfig$ free -h
              total        used        free      shared  buff/cache   available
Mem:           3.8Gi        2.5Gi        154Mi        15Mi        1.1Gi        1.0Gi
Swap:          2.0Gi        1.0Mi        2.0Gi
test@sat3210-virtual-machine: ~/fhirConfig$ sudo ufw status
[sudo] password for test:
Status: inactive
test@sat3210-virtual-machine: ~/fhirConfig$ sudo lsof -i :8090
COMMAND  PID USER  FD  TYPE DEVICE SIZE/OFF NODE NAME
docker-pr 3422 root   4u  IPv4  70982      0t0  TCP *:8090 (LISTEN)
docker-pr 3428 root   4u  IPv6  71768      0t0  TCP *:8090 (LISTEN)
test@sat3210-virtual-machine: ~/fhirConfig$
```

Figure 4: Port Utilization and Process Analysis

Figure 4 presents the detailed port utilization analysis performed using the `lsof` command to identify processes currently utilizing port 8090. The screenshot demonstrates that Docker proxy processes are actively managing both IPv4 and IPv6 network interfaces on

the target port, with process identifiers 3422 and 3428 respectively operating under root privileges. The analysis confirms that the HAPI-FHIR container is properly configured and actively listening for incoming connections on the designated port through Docker's network management infrastructure. This process analysis capability is essential for troubleshooting network connectivity issues, identifying port conflicts, and ensuring proper service availability in complex healthcare information exchange environments where multiple systems may compete for network resources.

4.0 Technical Architecture and Integration Considerations

The implemented HAPI-FHIR server represents a critical component within the broader Upper Peninsula Health Information Exchange architecture, serving as the central hub for healthcare data interoperability across multiple institutional boundaries. The server's strategic position enables seamless data exchange between the four regional hospitals including Aspirus, Portage Health, Baraga County Memorial Hospital, and Marquette General Hospital, while simultaneously supporting integration with external systems such as the Michigan Health Information Network (MiHIN) and the Western Upper Peninsula Health Department (WUPHD).

The Docker containerization approach provides significant advantages for healthcare information systems including consistent deployment characteristics across different infrastructure environments, simplified backup and disaster recovery procedures, enhanced security through process isolation, and streamlined maintenance and update procedures. The container architecture enables rapid scaling capabilities to accommodate varying data processing loads while maintaining consistent performance characteristics and security policies.

The H2 embedded database configuration provides an optimal balance between functionality and resource utilization for regional healthcare data exchange scenarios. While production deployments might benefit from external database systems for enhanced scalability and redundancy, the embedded approach simplifies initial deployment and reduces infrastructure complexity for development and testing environments. The database configuration supports full FHIR R4 compliance including complex resource relationships, search capabilities, and transaction processing essential for comprehensive healthcare data management.

4.1 Security Considerations and Compliance Framework

The HAPI-FHIR server implementation incorporates multiple layers of security controls appropriate for healthcare data processing environments. The Docker containerization provides process-level isolation, preventing potential security breaches from affecting the host system or other containerized applications. The container configuration implements least-privilege principles by running with minimal system permissions while maintaining necessary functionality for FHIR operations. The network configuration utilizes custom port mapping to provide security through obscurity while maintaining compatibility with standard FHIR client applications. The external configuration file approach enables dynamic security policy updates without requiring container rebuilding, facilitating rapid response to emerging security requirements or compliance mandates. The implementation supports comprehensive audit logging capabilities essential for healthcare data access monitoring and regulatory compliance reporting.

4.2 Performance Optimization and Scalability Considerations

The current implementation provides a solid foundation for healthcare data exchange operations while incorporating design elements that support future scalability requirements. The Docker containerization enables horizontal scaling through container orchestration platforms, allowing the system to accommodate increasing data volumes and user loads as the regional health information exchange network expands. The memory allocation analysis demonstrates adequate system resources for current operational requirements while providing headroom for increased utilization during peak data processing periods. The container configuration can be easily modified to adjust memory limits, CPU allocation, and other performance parameters based on operational experience and changing requirements.

4.3 Operational Procedures and Maintenance Framework

The implemented HAPI-FHIR server includes comprehensive operational procedures for routine maintenance, monitoring, and troubleshooting activities. The Docker container management approach simplifies backup procedures, update deployment, and system recovery operations essential for maintaining high availability in healthcare environments where system downtime can impact patient care delivery. The container lifecycle management procedures include standardized commands for starting, stopping, and restarting services, along with comprehensive logging capabilities for operational monitoring and troubleshooting. The external configuration management approach enables configuration changes without service interruption, supporting dynamic operational adjustments based on changing healthcare data exchange requirements.

5.0 Conclusion and Future Development Opportunities

The successful implementation of the HAPI-FHIR server on the UPHIE virtual machine represents a significant milestone in establishing comprehensive healthcare interoperability infrastructure for the Upper Peninsula region. The deployment demonstrates technical proficiency in modern containerization technologies, healthcare data standards implementation, and system administration practices essential for managing complex healthcare information systems. The implemented solution provides a robust foundation for supporting regional healthcare data exchange initiatives while incorporating design elements that facilitate future expansion and enhancement. The modular architecture supports integration with additional healthcare systems, implementation of advanced analytics capabilities, and deployment of specialized healthcare applications that leverage the FHIR data exchange infrastructure.

The technical skills demonstrated through this implementation including Docker containerization, system resource monitoring, network security assessment, and process management analysis are directly applicable to real-world healthcare information technology environments. These capabilities are essential for healthcare IT professionals responsible for implementing and maintaining interoperability solutions that support improved patient care coordination and public health initiatives. The successful completion of this assignment demonstrates readiness to contribute to healthcare information technology initiatives that leverage modern interoperability standards and containerization technologies to improve healthcare delivery and population health outcomes across diverse healthcare delivery environments.